

MODULES 26 & 27

Introduction to LangChain for Generative AI

Module 26 — Introduction to LangChain

- Lecture 1 — Getting Started with LangChain
- Lecture 2 — Why LangChain and Its Benefits

Module 27 — Getting Started with LangChain and OpenAI

- Lecture 1 — LangChain Website, LangSmith, LangGraph and LangServe
- Lecture 2 — Getting Started with OpenAI LLM

Module 26: Introduction to LangChain for Generative AI

Lecture 1: Getting Started with LangChain

LangChain is an open-source Python (and JavaScript) framework that makes it easy to build LLM-powered applications. Rather than writing boilerplate code to call APIs, manage prompts, and chain steps together, LangChain provides ready-made components for all of these.

What Problem Does LangChain Solve?

Building with LLMs in raw form requires handling many concerns:

- Prompt construction — formatting system messages, history, and user input correctly.
- LLM API calls — managing keys, retries, streaming, and model selection.
- Chaining steps — feeding the output of one step as the input to the next.
- Memory — storing conversation history so the model maintains context.
- Retrieval — fetching relevant documents from a database to inject into the prompt.
- Agents — letting the model decide which tools to call and in what order.

LangChain in one sentence

LangChain is the glue layer between your application and the LLM, providing modular, composable components for prompts, models, memory, retrieval, chains, and agents — so you build products, not boilerplate.

Installation and Setup

```
# Install core LangChain + OpenAI integration
pip install langchain langchain-openai langchain-community
pip install python-dotenv # for managing API keys

# .env file (never commit this to GitHub)
OPENAI_API_KEY=sk-...

# Load in Python
from dotenv import load_dotenv
import os
load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
```

Core LangChain Components

Component	What It Does Example
Models (LLMs / Chat)	Wrappers around OpenAI, Anthropic, Ollama, HuggingFace ChatOpenAI, OllamaLLM
Prompts	PromptTemplate, ChatPromptTemplate — reusable prompt builders Fill variables into templates
Output Parsers	Convert raw LLM text to structured types StrOutputParser, JsonOutputParser
Chains (LCEL)	Pipe components together with operator prompt llm parser
Memory	Store and retrieve conversation history ConversationBufferMemory
Retrievers / VectorDB	Fetch relevant docs from a knowledge base FAISS, Chroma + RAG
Agents & Tools	LLM decides which tool to call next Calculator, web search, SQL

Lecture 2: Why LangChain and Its Benefits

LangChain is not the only way to use LLMs, but it is the most widely adopted framework in the industry because it abstracts the hardest engineering problems and lets developers focus on product logic.

LangChain vs Raw API Calls

Task	Raw OpenAI API LangChain
Simple completion	5-10 lines of requests code 2-3 lines
Prompt templates	Manual f-string formatting PromptTemplate with variables
Multi-step chains	Manual variable passing prompt llm parser pipeline
Conversation memory	Manually append messages list ConversationBufferMemory
RAG pipeline	~100 lines (embed, store, retrieve, format) ~20 lines
Switching LLM	Rewrite all API calls Change one class name
Streaming output	Custom SSE handling .stream() on any chain

Key Benefits

- Model agnostic:** Switch between OpenAI, Anthropic, Google, Ollama, HuggingFace by changing one line.
- LCEL (LangChain Expression Language):** Chain components with | — readable, composable, supports streaming and async.
- Massive ecosystem:** 200+ integrations: vector DBs, document loaders, tools, retrievers.
- Production ready:** LangSmith for tracing/debugging, LangServe for API deployment, LangGraph for agents.
- Community:** Largest LLM framework community; actively maintained; extensive documentation.

When NOT to Use LangChain

LangChain adds abstraction overhead

Simple single-prompt app: raw OpenAI API is simpler and more transparent.

Maximum control needed: LangChain's abstractions can obscure what is happening.

Very high performance: direct API calls have lower latency than the LangChain wrapper.

Rule of thumb: use LangChain when you need chains, memory, RAG, or agents.

LangChain Ecosystem Overview

Package	Purpose
langchain	Core abstractions: chains, memory, agents, retrievers
langchain-openai	OpenAI-specific integrations (ChatOpenAI, OpenAIEmbeddings)
langchain-community	Community-contributed integrations (Ollama, HuggingFace, FAISS)
langchain-core	Base interfaces and LCEL primitives
langgraph	Graph-based stateful agent orchestration
langserve	Deploy LangChain chains as REST APIs (FastAPI-based)
langsmith	Observability: trace, debug, evaluate LLM app runs

Module 27: Getting Started with LangChain and OpenAI

Lecture 1: LangChain Website, LangSmith, LangGraph and LangServe

The LangChain ecosystem extends beyond the core framework into three production-grade tools: LangSmith for observability, LangGraph for complex agent workflows, and LangServe for deployment. Understanding each helps you choose the right tool at each stage.

LangChain Website — docs.langchain.com

- Official documentation: conceptual guides, API reference, how-to guides, tutorials.
- Hub (smith.langchain.com): browse and share prompt templates and chains.
- Integrations page: full list of 200+ LLM providers, vector stores, document loaders.

LangSmith — Observability and Debugging

LangSmith is a platform for tracing, debugging, evaluating and monitoring LLM applications. Every chain run can be logged automatically — showing inputs, outputs, latency, and token costs at each step.

```
# Enable LangSmith tracing - add to .env
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=ls___...
LANGCHAIN_PROJECT=my-project

# That's it - all LangChain runs are traced automatically
# View at: smith.langchain.com -> your project

# What you see in LangSmith per run:
# - Full prompt sent to LLM
# - Raw LLM response
# - Token counts (prompt + completion)
# - Latency at each chain step
# - Errors and retries
```

LangSmith Feature	Benefit
Run tracing	See every prompt, response, and intermediate step for every run
Datasets	Build evaluation datasets from production traces
Evaluators	Auto-evaluate chain outputs against ground truth or criteria
Feedback	Attach human ratings to runs for future fine-tuning
Monitoring	Dashboard: latency, cost, error rate over time

LangGraph — Stateful Agent Workflows

LangGraph models LLM applications as directed graphs where nodes are functions/chains and edges are transitions. Supports cycles (loops), conditional branching, and persistent state — enabling complex multi-step agent behaviour.

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

# Define state schema
class AgentState(TypedDict):
    messages: list
    next_action: str

# Define nodes (functions)
def call_llm(state): ...
def call_tool(state): ...
def should_continue(state): ...
```

```

    return 'tool' if state['next_action'] == 'tool' else END

# Build graph
graph = StateGraph(AgentState)
graph.add_node('llm', call_llm)
graph.add_node('tool', call_tool)
graph.add_edge('tool', 'llm') # after tool, back to LLM
graph.add_conditional_edges('llm', should_continue) # LLM decides next
graph.set_entry_point('llm')
app = graph.compile()

```

LangServe — Deploy as REST API

LangServe wraps any LangChain chain into a production FastAPI server with a single function call — including auto-generated docs and a playground UI.

```

# server.py
from fastapi import FastAPI
from langserve import add_routes
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

app = FastAPI(title='My LLM API')

prompt = ChatPromptTemplate.from_template('Tell me a joke about {topic}')
chain = prompt | ChatOpenAI() | StrOutputParser()

add_routes(app, chain, path='/joke')

# Run: uvicorn server:app --reload
# Auto-generated endpoints:
#   POST /joke/invoke      <- single request
#   POST /joke/stream      <- streaming response
#   GET  /joke/playground  <- browser UI

```

Lecture 2: Getting Started with OpenAI LLM in LangChain

This lecture builds practical code from first principles — connecting LangChain to OpenAI, creating prompt templates, building LCEL chains, and adding conversation memory.

Step 1 — Basic LLM Call

```

from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage

# Instantiate the model
llm = ChatOpenAI(
    model      = 'gpt-4o',
    temperature = 0.7,
    max_tokens  = 500
)

# Direct invocation — list of messages
messages = [
    SystemMessage(content='You are a helpful AI assistant.'),
    HumanMessage(content='What is LangChain in one sentence?')
]
response = llm.invoke(messages)
print(response.content)
print(f'Tokens used: {response.usage_metadata}')

```

Step 2 — Prompt Templates

```
from langchain_core.prompts import ChatPromptTemplate

# Define a reusable template with variables
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are an expert in {domain}. Answer concisely.'),
    ('human', '{question}'))

# Format the prompt with values
formatted = prompt.invoke({'domain': 'machine learning', 'question': 'What is overfitting?'})
print(formatted.messages)  # shows the final messages list

# Common template types
from langchain_core.prompts import PromptTemplate
simple = PromptTemplate.from_template('Summarise this in 3 words: {text}')
```

Step 3 — LCEL Chains (The | Operator)

LangChain Expression Language (LCEL) lets you compose components into pipelines using the | (pipe) operator — similar to Unix pipes.

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model='gpt-4o')
prompt = ChatPromptTemplate.from_template(
    'Explain {topic} in simple terms for a beginner.'
)
parser = StrOutputParser()  # extracts .content string from AIMessage

# Build chain with | operator
chain = prompt | llm | parser

# Invoke
result = chain.invoke({'topic': 'self-attention in Transformers'})
print(result)  # clean string output

# Streaming
for chunk in chain.stream({'topic': 'gradient descent'}):
    print(chunk, end='', flush=True)

# Batch (parallel execution)
results = chain.batch([
    {'topic': 'neural networks'},
    {'topic': 'random forests'},
    {'topic': 'SVMs'}
])
```

Step 4 — Conversation Memory

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

llm = ChatOpenAI(model='gpt-4o')
store = {}  # session_id -> message history

def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]
```

```
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    MessagesPlaceholder(variable_name='history'),
    ('human', '{input}'))
])

chain = prompt | llm

# Wrap with message history
with_memory = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key = 'input',
    history_messages_key= 'history'
)

config = {'configurable': {'session_id': 'user_123'}}

r1 = with_memory.invoke({'input': 'My name is Alice.'}, config=config)
r2 = with_memory.invoke({'input': 'What is my name?'}, config=config)
print(r2.content) # 'Your name is Alice.'
```

Step 5 — Switching Models (Model Agnosticism)

```
# Switch from OpenAI to Ollama (local) by changing one import
from langchain_community.llms import Ollama
llm = Ollama(model='llama3') # runs locally via Ollama

# Switch to Anthropic Claude
from langchain_anthropic import ChatAnthropic
llm = ChatAnthropic(model='claude-3-5-sonnet-20241022')

# Switch to Google Gemini
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(model='gemini-1.5-pro')

# The same chain works with any of the above — just swap the llm variable
chain = prompt | llm | parser
```

LCEL best practices

Always end chains with `StrOutputParser()` to get clean string output.
 Use `.stream()` for any user-facing app — instant first token feels faster.
 Use `.batch()` for processing multiple items — runs in parallel automatically.
 Tag your runs: `chain.with_config({'run_name': 'my_chain'})` for LangSmith tracing.

End of Modules 26 & 27 Notes — Generative AI Course